```
    var claims = await Model.UserManager.GetClaimsAsync(user);
    if (claims.Any())
    {
        <h5>@user.UserName</h5>
        <table class="table-striped col-12">
            <tr>
                <th>Type</th>
                <th>Value</th>
                <th>Issuer</th>
            </tr>
            @foreach (var claim in claims)
            {
                <tr>
                    <td>@claim.Type</td>
                    <td>@claim.Value</td>
                    <td>@claim.Issuer</td>
                </tr>
            }
        </table>
    }
}
```

It is worth noting that the preceding code includes something you should avoid for your production application. That is an example of the N + *1 problem*—so called because the code makes one database call for the users (in the `OnGetAsync` method) and then *N* further calls to the database, where *N* represents the number of results retrieved in the first call. Each time `GetClaimsAsync` is executed, a database query is issued. Depending on your application, that could hurt performance terribly if you have a lot of data and/or concurrent users. If you ever find yourself needing to iterate over the claims of all users, you should consider writing your own SQL to obtain all the relevant data in one call.

When you run this page for the first time, all the database calls are made, but there is no data to be displayed, so you will only be presented with the button inviting you to add a new claim. It goes nowhere at the moment because you have yet to create the page.

Add a new page to the ClaimsManager folder named Assign. This page will feature a list of users in a select list and inputs for a claim type and value.